Strapi CMS

The PhysioGuidance content management system — content types, i18n, admin roles, publishing workflow, media uploads, and how the Next.js frontend fetches content.

PREPARED FOR

PhysioGuidance Content Team

DATE

2026-08-17

ADMIN URL

cms.physioguidance.org/admin

PHYSIOGUIDANCE

Riyadh · Saudi Arabia
me-central2 · GCP Cloud Run

Table of contents

1. What Strapi CMS is and what it powers
2. Deployment topology
3. Repository layout
4. Content types
5. Reusable components and dynamic zones
6. Internationalisation (EN and AR)
7. Admin roles and access
8. Content workflow — draft, publish, unpublish
9. Media and uploads (Google Cloud Storage)
10. Frontend integration — how Next.js reads Strapi
11. Blog implementation (react-markdown)
12. Seed script and initial content
13. Backup and restore
14. Common questions
15. Where to look next

---

1. What Strapi CMS is and what it powers

Strapi is a self-hosted, open-source headless CMS running Strapi **v5.35.0** on Node 20. It stores every piece of editable public-facing content: hero copy, feature lists, blog posts, instructor profiles, testimonials, partner logos, FAQ entries, marketing pages, navigation labels, footer links, SEO metadata, and Arabic translations of all of the above.

The Next.js frontend at `physioguidance.org` reads Strapi over HTTP at build-time and request-time. Content authors edit inside Strapi's admin panel; the frontend picks up the new content on the next request. There is no rebuild step between publishing and going live — the frontend fetches with `cache: 'no-store'` so an editor's "Publish" click is visible on refresh.

Deployed commit: `d396b040`.

What Strapi does **not** own:

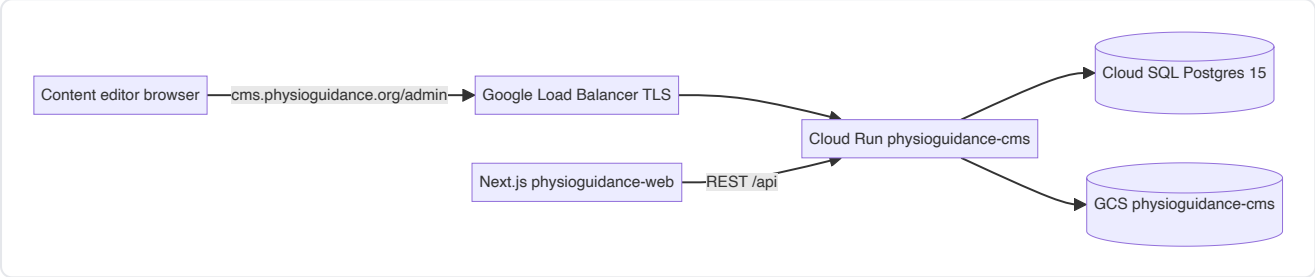
- Course catalog, modules, quizzes, enrolments — those live in the NestJS API and its Postgres tables (business data, not editorial copy).
- User accounts, sessions, roles for the learner platform — those live in the NestJS API.

- Payment records, invoices, refunds — NestJS API.
- Certificate PDFs — generated by the NestJS API and stored in the private assets bucket.

Strapi owns the **words on the marketing site and the blog**. The learner-facing platform gets its data from the NestJS backend.

---

2. Deployment topology



Cloud Run service: `physioguidance-cms` **Region:** `me-central2` (Dammam) **Custom domain:** `https://cms.physioguidance.org` **Admin panel:** `https://cms.physioguidance.org/admin` **Public REST API:** `https://cms.physioguidance.org/api` **Health check:** `https://cms.physioguidance.org/_health`

Runtime configuration (from `infra/config.sh`):

Setting	Value
Service name	`physioguidance-cms`
Port	`1337`
Memory	`512Mi`
CPU	`1`
Min instances	`0` (scales to zero)
Max instances	`2`
Runtime service account	`strapi-runtime@nelc-elearning-platform-479319.iam.gserviceaccount.com`
Database	Cloud SQL Postgres 15 — instance `physioguidance-db-me`, database `strapi`, user `apiuser`
DB connection	Cloud Run built-in Cloud SQL socket at `/cloudsql/PROJECT:REGION:INSTANCE`
Upload bucket	`physioguidance-cms` (public, uniform bucket-level access)
VPC connector	`physio-vpc-connector-me` (all egress via VPC)

Secrets are pulled from GCP Secret Manager at container start:

- `STRAPI_APP_KEYS` — session cookie signing keys
- `STRAPI_API_TOKEN_SALT` — API-token hashing salt
- `STRAPI_ADMIN_JWT_SECRET` — admin panel JWT signing key
- `STRAPI_TRANSFER_TOKEN_SALT` — data-transfer token salt
- `STRAPI_JWT_SECRET` — users-permissions JWT signing key
- `STRAPI_DATABASE_PASSWORD` — Postgres password for the `apiuser` role

Deployment happens through `infra/strapi/deploy.sh` (multi-stage Docker build, push to Artifact Registry `me-central2-docker.pkg.dev/nelc-elearning-platform-479319/physioguidance`, then `gcloud run deploy`). The Dockerfile (`infra/strapi/Dockerfile`) builds the admin panel with `npm run build`, prunes dev dependencies, and copies the compiled `dist/` to a minimal `node:20-alpine` runtime.

---

3. Repository layout

The Strapi project lives at `strapi/` in the repo root.

```
strapi/
├── config/                    # Server, DB, admin, plugins, middlewares, i18n
│   ├── admin.ts               # JWT + API token secrets from env
│   ├── api.ts                 # REST defaults (limit 25, max 100, withCount true)
│   ├── database.ts            # SQLite / MySQL / Postgres switch
│   ├── middlewares.ts         # CSP hardened for GCS images
│   ├── plugins.ts             # i18n + GCS upload provider
│   └── server.ts              # HOST / PORT / APP_KEYS
├── src/
│   ├── admin/                 # Admin panel customisations
│   ├── api/                   # 26 content types (see §4)
│   ├── components/            # Reusable components — blocks / elements / shared
│   ├── extensions/            # Empty — reserved for plugin extensions
│   └── index.ts               # Bootstrap — auto-creates 'ar' locale + Public role perms
├── database/migrations/       # Empty — Strapi manages schema from content-type JSON
├── public/                    # Static assets (favicon)
├── scripts/                   # Seed scripts (see §12)
├── types/                     # Generated Strapi types
└── package.json               # Strapi 5.35.0 + plugins
```

Deployment glue lives at `infra/strapi/`:

- `Dockerfile` — multi-stage Node 20 Alpine build
- `deploy.sh` — Cloud Run deploy driver

---

4. Content types

Strapi has **26 content types**, split between two **collection types** with many entries and **single types** that hold one document per locale.

Collection types (many entries per locale)

Type	Purpose
`blog-post`	Blog articles with rich content, category, main image, tags, SEO
`blog-category`	Categories referenced by blog posts
`instructor`	Instructor profile cards (name, title, photo, bio)
`testimonial`	Student and partner testimonials
`partner`	Partner logos and names
`navigation-link`	Header and footer navigation entries
`profession`	Professions with nested specialties for the trainer application form
`page`	Dynamic marketing pages created purely from the CMS (slug + dynamic-zone blocks)

Single types (one document per locale)

Each single type below holds either flat scalar fields or a dynamic-zone `blocks` field that composes any of the ~50 reusable block components (§5).

Type	Powers this frontend route
`home-page`	`/`
`about-page`	`/about`
`become-trainer-page`	`/become-trainer`
`contact-page`	`/contact`
`feedback-page`	`/feedback`
`partners-page`	`/partners`
`instructors-page`	`/instructors`
`certifications-page`	`/certifications`
`nelc-page`	`/nelc`
`learning-policies-page`	`/policies`
`privacy-policy-page`	`/privacy`
`terms-page`	`/terms`
`faq-page`	`/faq`
`support-page`	`/support`
`search-page`	`/search`
`courses-page`	`/courses` (labels and hero copy)
`course-detail-page`	`/courses/[slug]` (labels and empty states)
`blogs-page`	`/blogs` (section titles and empty states)
`site-setting`	Global site name, logo, footer settings

`blog-post` schema — reference

`strapi/src/api/blog-post/content-types/blog-post/schema.json`:

- `title` — string, required, localized
- `slug` — string, required, unique (shared across locales)
- `excerpt` — text, localized
- `content` — richtext (Markdown), localized
- `readTimeMinutes` — integer, default 5
- `featured` — boolean, default false
- `publishedDate` — date
- `authorName` — string, localized

- `tags` — JSON
- `category` — many-to-one relation → `blog-category`
- `mainImage` — single media, images only
- `order` — integer, default 0
- `seo` — component `shared.seo` (metaTitle, metaDescription, ogImage, keywords, canonicalUrl, noindex, twitterCard)

`blog-post` and `blog-category` both have Draft & Publish enabled and are localised.

---

5. Reusable components and dynamic zones

Components live in `strapi/src/components/` and split into three folders:

- **`blocks/`** — 51 large block components that appear in dynamic zones (`hero-banner` , `featured-courses-grid` , `cta-banner` , `image-banner-block` , `stats-numbers-block` , `cards-grid` , `video-embed` , `logo-strip` , `faq-accordion` , `accordion-block` , `contact-form-block` , `feedback-form-block` , `trainer-form-block` , `certification-cards-block` , `accreditation-badges-block` , `announcement-banner-block` , `before-after-block` , `blog-posts-grid-block` , `breadcrumb-block` , `comparison-table-block` , `compliance-badge-block` , `countdown-timer-block` , `course-preview-block` , `cpd-calculator-block` , `curriculum-outline-block` , `download-resources-block` , `image-gallery-block` , `instructor-spotlight-block` , `learning-outcomes-block` , `newsletter-block` , `pdf-viewer-block` , `pricing-table-block` , `related-pages-block` , `section-header-block` , `social-feed-block` , `spacer-block` , `student-progress-block` , `success-stories-block` , `tabs-block` , `timeline-block` , `treatment-gallery-block` , `upcoming-events-block` , `rich-text-block` , `partners-strip` , `instructors-grid` , `featured-section-block` , `testimonials-carousel` , `live-counter-block` , `policy-sections-block` , `anchor-links-block` , `instructor-reviews-block`).
- **`elements/`** — smaller repeatable sub-components used inside blocks (`accordion-item` , `card-item` , `curriculum-module` , `feature-item` , `stat-item` , `tab-item` , `timeline-item` , `download-item` , `event-item` , `cpd-tier` , `specialty-option` , and 20+ more).
- **`shared/`** — cross-cutting components: `seo` (SEO metadata), `faq-item` , `policy-section` , `search-term` , `support-category` .

Every single-type page with a `blocks` dynamic zone lets editors pick and reorder any of the 51 block components. This is the source of the site's flexibility — a page's layout is composed in the admin panel rather than hardcoded in Next.js.

The **`shared.seo` component** is attached to every content type that renders as a URL (blog posts, blog categories, marketing pages). Fields: `metaTitle` , `metaDescription` , `keywords` , `ogImage` ,

`ogTitle`, `ogDescription`, `canonicalUrl`, `noindex`, `twitterCard` (`summary` or `summary_large_image`).

---

6. Internationalisation (EN and AR)

Locales are configured in `strapi/config/plugins.ts`:

```
i18n: {
  enabled: true,
  config: {
    defaultLocale: 'en',
    locales: ['en', 'ar'],
  },
}
```

The Arabic locale is auto-created on first boot by `strapi/src/index.ts::bootstrap()` — if the `ar` locale is missing, it is created programmatically so no manual admin step is needed after a fresh deploy.

How editors work with locales

At the top-right of every content-type edit page the admin panel shows a **locale switcher** (English / Arabic). Fields marked `localized: true` in the schema (nearly every text field) show a per-locale value; each translation is its own document under the hood but they share `documentId`, `slug`, and the same media relations.

Publishing English does not publish Arabic. Each locale is a separate publish action.

How the frontend requests a locale

Every request adds `?locale=en` or `?locale=ar` as a **query parameter**, never in a POST body. Examples from `frontend/src/lib/strapi/api.ts`:

- `GET /api/home-page?locale=en&populate[blocks][on][blocks.hero-banner][populate]=*`
- `GET /api/blog-posts?locale=ar&populate=*&sort[0]=publishedDate:desc`

Strapi returns only the localised value for that locale. If a locale is missing for a field, Strapi returns `null`; the frontend translation-fallback logic handles it.

---

7. Admin roles and access

Strapi ships two independent role systems:

Admin panel roles (for editors, at `cms.physioguidance.org/admin`)

These control who can log into the CMS admin panel and what they can edit.

Role	Typical assignee	Powers
Super Admin	Platform owner	Full control — content, users, roles, settings, plugins, API tokens, media library
Editor	Content lead	Create, read, update, publish, and delete all content types across every locale. Cannot manage users, roles, or settings
Author	Blog writer, translator	Create, read, and update only their own content. Cannot publish — needs an Editor or Super Admin to review and hit Publish

New admin users are invited from **Settings** → **Administration Panel** → **Users** → **Invite new user**. The invited email receives a one-time link to set a password. All admin auth is username/password with a JWT session; MFA is available in Strapi Enterprise Edition and is a future upgrade path.

Users-permissions roles (for the public REST API)

These control what the **REST API** returns to unauthenticated or token-authenticated callers. Managed under **Settings** → **Users & Permissions** → **Roles**.

Role	Purpose
Public	The role unauthenticated API calls run as. Enabled only for read-only endpoints the frontend needs without a token: `blog-post/find`, `blog-post/findOne`, `instructor/find`, single-type page reads. Bootstrapped in `strapi/src/index.ts` to also allow `profession/find` and `profession/findOne` so the Become-a-Trainer form's cascade dropdown works without auth
Authenticated	Reserved for future authenticated site visitors — currently unused

The Next.js frontend also holds an **API token** (`STRAPI_API_TOKEN` env var) that gives it full read access; that token is sent as `Authorization: Bearer <token>` on every server-side fetch. Tokens are minted under **Settings** → **API Tokens** and revoked from the same screen.

---

8. Content workflow — draft, publish, unpublish

Every content type is created with `draftAndPublish: true`. An editor's lifecycle for a piece of content:



Save vs Publish. *SAVE* records changes to the draft; the live site does not change. *Publish* promotes the draft to the published version, and the next frontend fetch (a browser refresh) shows it. Because the frontend fetches with `cache: 'no-store'`, there is no CDN-cache lag — publishing is visible on the next request.

Unpublish. Opens the record on the admin panel and click **Unpublish**. The record returns to draft state and the frontend fetch stops returning it (public `find` calls return only published entries in production).

Preview drafts on the frontend. In development, the Next.js fetch layer automatically appends `status=draft` (see `frontend/src/lib/strapi/client.ts`), so editors running the frontend locally see unpublished content. Production always sees only the published version.

Deleting. Delete is destructive and cannot be undone from the admin panel — restore requires a database restore (§13). Prefer **Unpublish** when hiding content is enough.

---

9. Media and uploads (Google Cloud Storage)

Uploaded images, videos, and files never touch the Cloud Run container filesystem — the filesystem is ephemeral, so a file saved to local disk would vanish on the next cold start, deploy, or auto-scale event.

Strapi is configured with the community GCS upload provider (`strapi/config/plugins.ts`):

```
upload: {
  config: {
    provider: '@strapi-community/strapi-provider-upload-google-cloud-storage',
    providerOptions: {
      bucketName: env('GCS_BUCKET_NAME') ?? 'physioguidance-cms',
      basePath: env('GCS_BASE_PATH') ?? '',
      baseUrl: env('GCS_BASE_URL'),
      publicFiles: (env('GCS_PUBLIC_FILES') ?? 'true') === 'true',
      uniform: (env('GCS_UNIFORM_ACCESS') ?? 'true') === 'true',
    },
  },
}
```

Bucket: `gs://physioguidance-cms` in `me-central2`, uniform bucket-level access, publicly readable objects. URLs Strapi returns look like `https://storage.googleapis.com/physioguidance-cms/uploads/<hash>.jpg`.

Authentication. The Cloud Run service runs as the `strapi-runtime` service account, which has `roles/storage.objectAdmin` on the `physioguidance-cms` bucket only (least privilege granted in

`least-privilege-service-accounts-for-amna.sh`). The upload provider uses Application Default Credentials from the metadata server — no key file is required in production.

CSP allowlist. `strapi/config/middlewares.ts` extends the default Strapi security middleware so the admin panel preview can render images and video from `https://storage.googleapis.com`.

Image formats. Strapi's built-in image processor generates responsive variants: `thumbnail`, `small`, `medium`, `large`. The frontend prefers a specific variant per surface — for example the blog thumbnail uses `formats.medium.url` and falls back to the original.

Size limits. Strapi's default upload limit is 200 MB per file. Cloud Run has a 32 MB request-body ceiling for uncompressed uploads — larger files should be uploaded directly to GCS by the backend, not through the CMS admin panel.

Supported types. Images (jpg, png, webp, gif, svg), videos (mp4, webm), audio (mp3, wav), documents (pdf), archives (zip). Per-content-type schemas restrict which media types are allowed (blog post main images are `["images"]` only).

---

10. Frontend integration — how Next.js reads Strapi

The frontend integration lives at `frontend/src/lib/strapi/`:

```
frontend/src/lib/strapi/
├── index.ts        # Barrel — re-exports client + api + types
├── client.ts       # fetchStrapi() + getStrapiMediaUrl() — plain module
├── api.ts          # 'use server' — 54 typed page-fetcher functions
├── types.ts        # TypeScript interfaces for every content type
└── block-utils.ts  # Dynamic-zone block helpers
```

The `'use server'` pattern

`api.ts` starts with the `'use server'` directive. Every exported function in that file is a Next.js **Server Function** — it can only execute on the server, and importing it into a Client Component and calling it during initial render throws:

> Server Functions cannot be called during initial render

`index.ts` re-exports everything from `api.ts` and `client.ts`. Because the barrel re-exports the server-function file, **any import from `@/lib/strapi/*` in a Client Component that resolves to a symbol from `api.ts` will fail at render time**.

The `getStrapiMediaUrl` rule

`getStrapiMediaUrl(url)` is a pure string-manipulation helper — no server-only logic. But it lives in `client.ts` which `api.ts` re-exports, and `api.ts` also re-exports the helper alongside its server functions. So importing it from the barrel drags the whole server-function module in.

Always import `getStrapiMediaUrl` from `@/lib/strapi/client` — never from `@/lib/strapi`. Server Components can safely use the barrel; Client Components must go direct to `client.ts`. This is written up in the project MEMORY doc and applies to every new component.

How a page fetches

Typical Server Component pattern:

```
// frontend/src/app/[locale]/blogs/[slug]/page.tsx
import { getBlogDetailData, getBlogBySlug, getSiteSettings, getStrapiMediaUrl }
  from "@/lib/strapi";

const { locale, slug } = await params;
const [post, settings] = await Promise.all([
  getBlogBySlug(slug, locale).catch(() => null),
  getSiteSettings(locale).catch(() => null),
]);
```

Each fetcher builds a URL like:

`GET /api/blog-posts?filters[slug][$eq]=<slug>&locale=en&populate=*`

then returns typed data or `null` (never throws — graceful degradation is preferred so a broken CMS never black-holes a page).

Caching

`fetchStrapi()` sets `cache: 'no-store'` on every request. There is no ISR window; every page render fetches fresh. Strapi is co-located in `me-central2` so intra-region latency is 20–50 ms, and the trade-off is worthwhile because editors expect "Publish" to be instant.

API-token auth

`STRAPI_API_TOKEN` is set as a Cloud Run env var on the `physioguidance-web` service (sourced from Secret Manager). Every server-side fetch sends `Authorization: Bearer <token>`. The token has full read access — it never mutates.

---

11. Blog implementation (react-markdown)

The blog was migrated off Sanity onto Strapi. The migration replaced two things:

- **Content storage.** Sanity `document` types → Strapi collection types `blog-post` and `blog-category`.
- **Rich-text renderer.** Sanity's `@portabletext/react` block renderer → **`react-markdown`**, because Strapi's `richtext` field is Markdown.

Blog content stored in Strapi is Markdown. On the frontend it renders through `react-markdown` in `frontend/src/components/blogs/`. The same library is reused across the site — `PolicyPageLayout.tsx`, `TabsBlock.tsx`, `InstructorSpotlightBlock.tsx`, `RichTextBlock.tsx`, and `FAQAccordionBlock.tsx` all use `react-markdown`, so authors get consistent Markdown rendering everywhere.

Image URL pattern for blog posts

The blog post schema's `mainImage` is a single media field. Strapi returns it with an `url` plus `formats.thumbnail`, `formats.small`, `formats.medium`, `formats.large` (all with their own URLs). The frontend prefers the `medium` variant and falls back to the original:

```
getStrapiMediaUrl(post.mainImage?.formats?.medium?.url || post.mainImage?.url)
```

TypeScript note for react-markdown `img`

When passing a custom `img` renderer to `react-markdown`, the `src` prop is typed loosely; wrap it with `String(src || "")` before handing to Next.js's `<Image>` to satisfy the type checker:

```
<ReactMarkdown components={{
  img: ({ src, alt }) => <img src={String(src || "")} alt={alt ?? ""} />
}}>
  {post.content}
</ReactMarkdown>
```

Blog SEO

Every blog post has a `seo` component (`shared.seo`) populated per locale. `generateMetadata` in the Next.js page reads `post.seo` and falls back to `post.title`, `post.excerpt`, and `post.mainImage` when SEO fields are blank.

---

12. Seed script and initial content

Initial content is seeded by Node scripts under `strapi/scripts/`:

Script	Purpose
`seed-content.js`	Full site content seed — home, about, become-trainer, policies, contact, feedback, certifications, NELC, instructors, partners, testimonials, blog posts, blog categories, navigation, site settings
`seed-blocks.js`	Populates dynamic-zone `blocks` for page-level single types
`seed-support-page-only.js`	Repopulates just the support page (used when iterating on that surface)
`seed-courses-page-labels.js`	Repopulates the `courses-page` labels only
`migrate-flat-seo-to-component.js`	One-shot migration script — moved legacy flat `metaTitle` / `metaDescription` / `ogImage` fields into the shared `shared.seo` component

Running a seed script

Requirements:

1. Strapi must be running (locally `npm run develop`, or the production URL).
2. An **API token with full access** must exist. Create one at **Settings → API Tokens → Create new API Token** (`Full access` type). Copy the token.
3. Set `STRAPI_URL` and `STRAPI_API_TOKEN` and run:

```
STRAPI_URL=https://cms.physioguidance.org \
STRAPI_API_TOKEN=<token> \
node strapi/scripts/seed-content.js
```

Conventions the scripts follow

The seed scripts encode the correct Strapi v5 REST conventions — reference them when writing new tooling:

- **Locale as query parameter, never in body.** POST creates use `?locale=en`; PUT single-type updates use `?locale=ar`. Body carries only the field values.
- **Single types use PUT** (not POST). The endpoint is `PUT /api/home-page?locale=en`. Strapi upserts.
- **Collection types use POST** with `?locale=en` to create, then a separate publish call to promote from draft to published.
- **Relations are set by numeric `id`, not `documentId`.** For example a blog post's `category` field is `{ category: 42 }`, not the category's `documentId`.
- `populate[field]=true` for a single-level relation; `populate[field][populate]=*` for nested relations and media inside a relation.

- `pagination[pageSize]=100` on every listing request when the seed needs to enumerate everything (default page size is 25). The `deleteAllEntries` helpers query both locales AND both statuses (`published` + `draft`) with pagination to catch every row.

Cleaning before re-seeding

The seed script's `deleteAllEntries()` helper deletes every entry across both locales and both statuses before inserting fresh data. Re-running the seed against a live CMS wipes existing content — always take a database snapshot first (§13).

---

13. Backup and restore

Strapi's data lives in two places, and both are backed up.

1. Content data — Cloud SQL `strapi` database

The Cloud SQL instance `physioguidance-db-me` holds Strapi's schema tables (`blog_posts` , `home_pages` , `instructors` , `components_shared_seos` , etc.) alongside the main app's `apiuser` -owned tables. Cloud SQL is protected by:

- **Automated daily backups** with 7-day retention.
- **Point-in-Time Recovery (PITR)** with 7-day transaction-log retention — restore to any second in the last week.
- **On-demand SQL exports** to the private `physioguidance-backups` GCS bucket via the admin dashboard "Backups" tab (30-day lifecycle rule, per `infra/scripts/setup-backups.sh`).

Recovery for Strapi content is the same procedure as the rest of the platform: restore Cloud SQL from its latest backup or PITR to a **clone** instance, verify the clone, then swap the primary. In-place restore is deliberately not exposed as a one-click action.

2. Uploaded media — `physioguidance-cms` GCS bucket

Uploaded images and files are objects in `gs://physioguidance-cms` . Because the bucket has uniform bucket-level access and public-read on objects, protection relies on GCS's built-in **object versioning** (enabled at bucket creation) plus retention policies. Deleted objects can be restored from the previous non-current version.

Restoring from a fresh deploy

If the CMS Cloud Run service itself is lost, redeploy from source with `infra/strapi/deploy.sh` . The service is stateless — it reconnects to Cloud SQL and GCS and picks up right where it left off. No content is stored on the container.

---

14. Common questions

Where do editors log in? `https://cms.physioguidance.org/admin` — the platform owner invites new users from **Settings** → **Administration Panel** → **Users**.

A change is published in the admin but the site still shows the old version. Confirm the change was **Published**, not just **Saved**. Check the top-right of the record — a green "Published" badge means live; an orange "Modified" or "Draft" means it hasn't been promoted. The frontend fetches with `cache: 'no-store'`, so once published, one hard refresh shows it.

How do I add a new marketing page? The single-type pages (about, contact, etc.) are hardcoded routes. For a new arbitrary page, use the **`page` collection type** — it has a slug and a dynamic-zone `blocks` field. The frontend catch-all route `frontend/src/app/[locale]/[...slug]/page.tsx` serves any published `page` entry at `/{locale}/{slug}`.

How do I translate a page into Arabic? Open the record, switch the top-right locale to **Arabic**, translate each localised field, save, and publish. The English record stays untouched — each locale publishes independently.

How do I upload a video too large for the admin upload widget? The 32 MB Cloud Run request-body ceiling applies to admin uploads. For larger files, upload directly to the `physioguidance-cms` bucket in the GCP Console (path `uploads/`), copy the public URL, and paste it into a rich-text or URL field. Or upload to the private assets bucket via the backend API for private files.

Can editors create new content types themselves? Only Super Admins can create/modify content types via the **Content-Type Builder** in development. In production the builder is disabled — schema changes ship through code (`strapi/src/api/*/content-types/*/schema.json`) and require a redeploy.

Where do I revoke an API token? Settings → API Tokens → click token → Delete. The frontend's `STRAPI_API_TOKEN` env var must then be updated in Secret Manager and the `physioguidance-web` Cloud Run service redeployed.

How do I add a new admin editor? Settings → Administration Panel → Users → Invite new user. Choose the **Editor** role for content-only access, **Author** for own-content-only access, **Super Admin** only for platform owners.

What happens if the CMS is offline? The frontend's `fetchStrapi()` catches errors and returns `null`; pages render with translation fallbacks from `frontend/messages/{locale}.json`. The site stays up with reduced editorial content until the CMS is back. The monitoring service at `physioguidance-cms/_health` alerts on this.

Where are the SEO fields on a page? Every page has a `seo` component group near the bottom of the edit form. Populate it per locale — the frontend's `generateMetadata` reads it and falls back to page title/excerpt/main image when a field is blank.

Is there a preview mode? Draft-visibility on the local frontend is automatic in development. A dedicated production preview URL (per-record, with a preview token) is a straightforward Strapi feature that can be enabled when the editorial workflow needs it.

How is the Arabic locale kept in sync? Each field with `localized: true` in the schema has its own value per locale. Publishing English does not publish Arabic and vice versa. The `strapi/src/index.ts` bootstrap guarantees the Arabic locale exists on first boot so editors never see a broken locale switcher.

---

15. Where to look next

For deeper context on adjacent systems referenced in this document:

- **System architecture** — `system-architecture.md` — where Strapi sits in the wider Cloud Run + Cloud SQL topology
- **GCP deployment** — `gcp-deployment.md` — Cloud Run service configuration, Cloud SQL, Artifact Registry, load balancer
- **CI/CD deployment** — `cicd-deployment.md` — the deploy pipeline that ships `infra/strapi/deploy.sh` on merges
- **Environment configuration (confidential)** — `environment-configuration-CONFIDENTIAL.md` — every Strapi env var and secret name
- **Security architecture** — `security-architecture.md` — service accounts, IAM, VPC, secrets rotation
- **PDPL compliance** — `pdpl-compliance.md` — data-residency posture (all Strapi data is in me-central2)
- **Ops runbook** — `ops-runbook.md` — on-call procedures if the CMS is unhealthy
- **Developer guide** — `developer-guide.md` — local development for the frontend, including running Strapi locally

Source of truth for anything in this document is the repository itself at commit `d396b040`:

- Strapi project: `strapi/`
- Deployment scripts: `infra/strapi/`
- Frontend integration: `frontend/src/lib/strapi/`
- Bootstrap logic: `strapi/src/index.ts`
- Content type schemas: `strapi/src/api/*/content-types/*/schema.json`
- Components: `strapi/src/components/{blocks,elements,shared}/`
- Seed scripts: `strapi/scripts/`